

XÂY DỰNG HỆ THỐNG SELF-EVOLVING AI AGENTS TRONG XỬ LÝ SỰ CỐ MẠNG VIỄN THÔNG

Nguyễn Thanh Hải

optimus1072005@gmail.com

Lĩnh vực Data Science & AI Engineering - Chương trình Viettel Digital Talent 2026

Nguyễn Duy Hưng[†]

hungnd1235@gmail.com

Nguyễn Duy Anh[†]

nguyenduyanh821998@gmail.com

Bùi Quang Hưng[†]

contact.hung.bq@gmail.com

[†]Phòng Giải pháp di động - Trung tâm Nghiên cứu Công nghệ & Chuyển đổi số - Tổng công ty Mạng lưới Viettel

1. GIỚI THIỆU CHUNG

Các mạng viễn thông hiện đại, đặc biệt trong bối cảnh 5G, trung tâm dữ liệu và mạng điều khiển bằng phần mềm, có quy mô lớn, nhiều lớp giao thức và phụ thuộc chặt chẽ giữa thiết bị, dịch vụ, cấu hình định tuyến, topology và chỉ số hiệu năng. Một sự cố nhỏ như sai ASN BGP, IP missing, link down, switch BMv2 dừng hoạt động hoặc rule SDN sai có thể gây mất kết nối diện rộng, suy giảm chất lượng dịch vụ và làm tăng chi phí vận hành. Trong thực tế, xử lý sự cố không chỉ là phát hiện bất thường, mà còn cần định vị thiết bị lỗi, xác định nguyên nhân (Root Cause Analysis - RCA) và đưa ra hướng khắc phục có thể kiểm chứng. Bài toán này khó vì triệu chứng quan sát được thường gián tiếp, chịu nhiễu và bắt nguồn từ nhiều tầng cấu hình khác nhau, gây khó khăn lớn cho kỹ sư vận hành.

Các cách tiếp cận truyền thống thường dựa trên luật thủ công, kinh nghiệm chuyên gia hoặc mô hình học máy tĩnh. Những phương pháp này khó thích ứng khi topology thay đổi, khi xuất hiện dạng lỗi mới hoặc khi dữ liệu vận hành có nhiễu. Sự phát triển của mô hình ngôn ngữ lớn (LLM) và AI Agent mở ra khả năng mới: agent có thể đọc mô tả sự cố, sử dụng công cụ chẩn đoán, suy luận nhiều bước và giải thích kết quả bằng ngôn ngữ tự nhiên. Các nghiên cứu gần đây về LLM cho chẩn đoán mạng nhấn mạnh tiềm năng kết hợp tri thức miền, log vận hành và công cụ kiểm chứng thay vì chỉ dựa vào câu trả lời một lượt của mô hình. Tuy nhiên, agent thông thường vẫn mang tính tĩnh; nếu prompt, bộ nhớ hoặc mô tả công cụ không được cập nhật thì hiệu quả chẩn đoán dễ suy giảm khi môi trường mạng thay đổi. Vì vậy, hệ thống cần học từ trace gọi công cụ, phản hồi chấm điểm và các case đã xử lý để điều chỉnh quy trình suy luận cho những sự cố tiếp theo.

Đề tài này nghiên cứu và xây dựng hệ thống **Self-Evolving AI Agents** cho xử lý sự cố mạng viễn thông trên nền tảng benchmark NIKA [10]. Khác với agent tĩnh, self-evolving agent khai thác chính lịch sử xử lý nhiệm vụ, phản hồi và kết quả đánh giá làm nguồn dữ liệu để cập nhật bộ nhớ, công cụ hoặc chiến lược suy luận [4]. Mục tiêu của đề tài là biến mỗi lần chẩn đoán thành một nguồn kinh nghiệm giúp agent cải thiện ở các lần sau. Cụ thể, hệ thống cần tiếp nhận dữ liệu từ môi trường mạng giả lập, gọi các công cụ kiểm tra trạng thái mạng, suy luận nguyên nhân gốc rễ, gửi kết quả đánh giá và cập nhật tri thức chẩn đoán dài hạn.

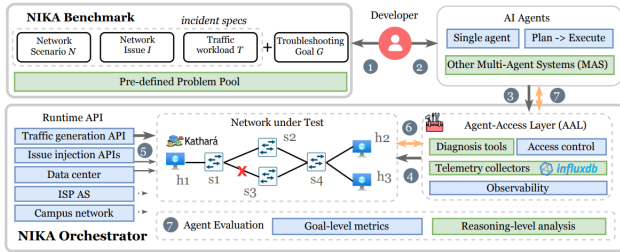
Ý nghĩa thực tiễn của sản phẩm nằm ở việc hỗ trợ kỹ sư vận hành thực hiện nhanh các bước lặp lại như kiểm tra kết nối, đọc cấu hình, đối chiếu bảng định tuyến, phân tích log và tổng hợp bằng chứng. Hệ thống không thay thế chuyên gia, mà đóng vai trò trợ lý chẩn đoán giúp khoanh vùng nguyên nhân, chuẩn hóa báo cáo sự cố và lưu lại kinh nghiệm đã kiểm chứng để tái sử dụng cho các ca tương tự. Đóng góp chính của sinh viên gồm:

- Thiết kế và hiện thực workflow agent chẩn đoán trên nền tảng NIKA/Kathará, tích hợp các dịch vụ MCP để agent tương tác với router, host, switch P4/BMv2, telemetry và hệ thống chấm điểm.
- Đề xuất hai mô-đun tự tiến hóa gồm **Procedural Memory** lấy cảm hứng từ Skill-Pro [6] và **Tool Refinement** lấy cảm hứng từ DRAFT [7], giúp agent lưu trữ kỹ năng chẩn đoán và tinh chỉnh tài liệu công cụ dựa trên trace thực tế.
- Xây dựng pipeline benchmark để so sánh baseline và agent tự tiến hóa trên NIKA, đánh giá theo detection, localization, RCA, số bước, số lần gọi công cụ, tool error rate, token và thời gian chạy.

2. NỘI DUNG VÀ PHƯƠNG PHÁP

2.1. Giới thiệu về benchmark NIKA

NIKA được giới thiệu như một network arena để đánh giá AI Agent trong bài toán xử lý sự cố mạng [10]. Benchmark này giải quyết một khoảng trống quan trọng: trước đây việc đánh giá agent chẩn đoán mạng thường thiếu môi trường chuẩn hóa, khó tái lập và tốn nhiều công sức vận hành. Như minh họa trong Hình 1, NIKA cung cấp các kịch bản mạng có thể replay, giao diện agent-network rõ ràng và tập incident được thiết kế để kiểm tra khả năng phát hiện lỗi, định vị thiết bị lỗi và xác định nguyên nhân gốc rễ. Vì vậy, NIKA phù hợp với mục tiêu của đề tài: xây dựng một agent không chỉ trả lời bằng ngôn ngữ tự nhiên, mà còn phải sử dụng công cụ, thu thập bằng chứng và cải thiện qua nhiều lần xử lý sự cố.



Hình 1. Tổng quan benchmark NIKA cho đánh giá AI Agent trong xử lý sự cố mạng.

2.2. Khung tổng quan hệ thống

Hệ thống được xây dựng quanh một agent chẩn đoán có khả năng tương tác với môi trường mạng thông qua các công cụ MCP. Ở mức tổng quan, môi trường mạng cung cấp topology, thiết bị và dữ liệu quan sát; agent sử dụng các công cụ để kiểm tra trạng thái mạng; bộ chấm điểm đánh giá kết quả cuối cùng; và hai mô-đun tự tiến hóa cập nhật tri thức chẩn đoán sau mỗi episode. Hai mô-đun này gồm **Procedural Memory** để lưu kinh nghiệm dưới dạng quy trình chẩn đoán và **Tool Refinement** để cải thiện tài liệu công cụ dựa trên trace thực thi.

2.2.1. Công cụ và công nghệ sử dụng

Hệ thống sử dụng NIKA làm benchmark chính để tạo các incident mạng và chấm điểm kết quả chẩn đoán. Kathará [5] được dùng để ảo hóa topology bằng container, giúp khởi động lab, tiêm lỗi và tái lập môi trường một cách nhất quán. MCP/FastMCP đóng vai trò lớp giao tiếp giữa agent và môi trường mạng; các thao tác như kiểm tra kết nối, đọc cấu hình, truy vấn telemetry hoặc nộp kết quả đều được chuẩn hóa thành công cụ. LangGraph/LangChain [2] được dùng để điều

phối workflow agent, trong đó các chiến lược chính gồm ReAct [11], Plan & Execute và Reflexion [9].

Ở tầng dữ liệu và mạng, hệ thống dùng InfluxDB để lưu telemetry, FRRouting [3] cho các kịch bản định tuyến và P4/BMv2 [1] cho các kịch bản data-plane lập trình được. Phần triển khai được viết bằng Python 3.12 và quản lý môi trường bằng uv. Các nhóm công cụ MCP chính gồm Kathará Base, FRR, BMv2/P4, Telemetry, Task Management và Tool Evolution; mỗi nhóm tương ứng với một phần quan sát hoặc điều khiển cần thiết trong quá trình agent xử lý sự cố.

2.2.2. Quy trình triển khai

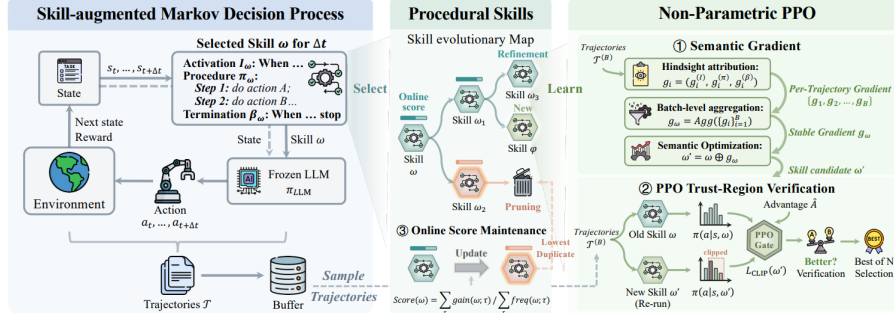
Quy trình triển khai bắt đầu từ việc chọn tập case trong benchmark, chuẩn bị cấu hình lab Kathará và xác định loại lỗi cần tiêm tương ứng với từng kịch bản. Mỗi lab được khởi động trong container riêng biệt để đảm bảo môi trường sạch và tái lập được giữa các episode. Sau khi lab sẵn sàng, lỗi được tiêm theo kịch bản định sẵn, agent nhận thông tin khởi đầu về topology và triệu chứng ban đầu rồi bắt đầu vòng chẩn đoán. Toàn bộ trace gồm suy luận, lời gọi công cụ, tham số và quan sát trả về đều được ghi lại để phục vụ chấm điểm và làm dữ liệu đầu vào cho hai mô-đun tự tiến hóa.

Luồng xử lý trong mỗi episode được tách thành hai pha. Trong **Diagnosis Phase**, agent có quyền dùng đầy đủ các công cụ quan sát để thu thập bằng chứng, kiểm tra giả thuyết và loại trừ nguyên nhân không phù hợp. Trong **Submission Phase**, agent chỉ còn công cụ nộp kết quả gồm `is_anomaly`, `faulty_devices` và `root_cause_name`, nhờ đó tránh tình trạng vừa chẩn đoán vừa thay đổi kết luận một cách thiếu kiểm soát. Với baseline, mỗi episode hoàn toàn độc lập và agent không mang kinh nghiệm nào từ trước. Với evolved agent, bộ nhớ kỹ năng và tài liệu công cụ đã được tinh chỉnh từ các episode trước được nạp vào ngữ cảnh trước khi bắt đầu case mới; sau khi episode kết thúc, **Procedural Memory** và **Tool Refinement** tiếp tục được cập nhật từ trace và kết quả chấm điểm.

2.3. Mô-đun Procedural Memory

Procedural Memory chuyển kinh nghiệm chẩn đoán từ dạng lịch sử trajectory dài sang các kỹ năng tái sử dụng [6]. Thay vì đọc lại toàn bộ trajectory cũ, hệ thống rút gọn kinh nghiệm thành các đơn vị kỹ năng có điều kiện kích hoạt, quy trình thực thi và điều kiện dừng rõ ràng. Như Hình 2, trace sau episode được dùng để sinh hoặc tinh chỉnh kỹ năng, giúp định hướng chuỗi hành động cho các episode tiếp theo.

Mỗi kỹ năng gồm ba thành phần: *Activation condition* xác định tình huống kích hoạt (như mất kết nối, lỗi định tuyến hoặc sự cố P4); *Execution procedure* chỉ ra chuỗi kiểm tra ưu tiên (reachability,



Hình 2. Quy trình học kỹ năng tái sử dụng cho mô-đun **Procedural Memory**.

config, routing table); và *Termination condition* xác định thời điểm dừng khi đủ bằng chứng thiết bị lỗi hoặc giả thuyết bị loại bỏ, giúp định hướng trực tiếp quá trình ra quyết định.

Tại mỗi incident, hệ thống truy xuất các kỹ năng phù hợp nhất dựa trên triệu chứng và topology ban đầu. Bộ chọn memory ưu tiên các kỹ năng có hiệu quả lịch sử cao để nạp vào prompt. Để xếp hạng, hệ thống duy trì bảng thống kê tần suất sử dụng, tỷ lệ thành công và chi phí tài nguyên của từng kỹ năng. Kỹ năng chỉ đóng vai trò định hướng hành vi: agent vẫn chủ động thực thi công cụ và có thể dừng kỹ năng nếu quan sát thực tế không còn khớp với điều kiện kích hoạt ban đầu.

Sau mỗi episode, hệ thống dùng phản hồi chấm điểm để cập nhật bộ nhớ. Trajectory thành công dùng để tạo hoặc tinh chỉnh kỹ năng; trajectory thất bại dùng để điều chỉnh điều kiện kích hoạt hoặc các bước gây nhiễu. Cơ chế semantic gradient tạo ra các sửa đổi, được kiểm duyệt qua PPO gate phi tham số [8] để đảm bảo chất lượng. Bộ nhớ cũng duy trì điểm chất lượng theo thời gian để lọc bỏ các kỹ năng trùng lặp hoặc kém hiệu quả. Các kỹ năng dưới ngưỡng chất lượng hoặc ít được gọi sẽ bị loại bỏ để duy trì kích thước bộ nhớ tối ưu.

2.4. Mô-đun Tool Refinement

Tool Refinement xử lý một nguồn lỗi quan trọng khác của agent: tài liệu mô tả công cụ có thể đầy đủ với kỹ sư vận hành nhưng lại thiếu các chi tiết cần thiết để LLM ra quyết định chính xác [7]. Trong các bài toán chẩn đoán mạng phức tạp, việc thực thi công cụ phụ thuộc lớn vào loại thiết bị, tên container, giao thức đang chạy và định dạng đầu ra. Nếu tài liệu thiếu các tiền điều kiện rõ ràng hoặc ví dụ cụ thể, agent dễ gặp lỗi gọi sai thiết bị, sai tham số hoặc hiểu sai kết quả trả về. Do đó, mô-đun này tập trung tinh chỉnh tài liệu mô tả công cụ bằng cách bổ sung chi tiết vận hành thực tế mà không cần sửa đổi mã nguồn gốc của công cụ, giúp tối ưu hóa hiệu năng chẩn đoán.

Bảng 1. Thuật toán học của mô-đun **Tool Refinement**

Input	Tập tài liệu công cụ $\mathcal{D}$, số vòng I , ngưỡng tương đồng ϕ , ngưỡng dừng τ .
Output	Tập tài liệu đã tinh chỉnh $\tilde{\mathcal{D}}$.

```

1  $\tilde{\mathcal{D}} \leftarrow \emptyset$ .
2 for mỗi tài liệu  $t \in \mathcal{D}$  do
3   for  $i = 1 \dots I$  do
4     Explorer sinh tình huống khám phá  $e_i$ .
5     Nếu  $e_i$  quá giống lịch sử theo  $\phi$ , sinh lại truy vấn khác.
6     Thực thi công cụ và ghi nhận kết quả  $r_i$ .
7     Analyzer rút lỗi tài liệu và gợi ý sửa đổi  $s_i$ .
8     Rewriter cập nhật  $t_i$  và hướng khám phá tiếp theo.
9     Nếu độ thay đổi  $\Delta(t_{i-1}, t_i) > \tau$ , dừng vòng học.
10  end for
11   $\tilde{\mathcal{D}} \leftarrow \tilde{\mathcal{D}} \cup \{t_i\}$ .
12 end for
13 return  $\tilde{\mathcal{D}}$ .
```

Quy trình học được mô tả trong Bảng 1. Explorer đóng vai trò tạo các tình huống sử dụng công cụ từ tài liệu hiện tại và lịch sử khám phá. Để đảm bảo độ bao phủ rộng, Explorer áp dụng ràng buộc tương đồng ϕ : nếu một tình huống mới quá giống lịch sử, hệ thống yêu cầu sinh lại truy vấn khác, ghi nhận toàn bộ tham số, kết quả thực thi và lỗi phát sinh làm nguồn dữ liệu huấn luyện phong phú. Việc đa dạng hóa các kịch bản này giúp mô hình phát hiện ra các trường hợp biên (edge cases) dễ gây lỗi cho hệ thống mạng thực tế.

Analyzer sau đó đối chiếu kết quả thực thi để phát hiện khoảng cách giữa hành vi thực tế và tài liệu, từ đó sinh các đề xuất chỉnh sửa cụ thể. Rewriter kết hợp các đề xuất này để viết lại tài liệu mới và định hướng Explorer ở vòng tiếp theo cho đến khi tài liệu đạt trạng thái hội tụ τ . Cách thiết kế này đảm bảo cải thiện hiệu quả chẩn đoán nhưng vẫn giữ ranh giới an toàn tuyệt đối cho hệ thống mạng.

3. THỰC NGHIỆM

3.1. Thiết lập thực nghiệm

Toàn bộ thực nghiệm được thực hiện trên hệ thống trang bị CPU 12 nhân và 16 GB RAM. Các mô hình ngôn ngữ lớn được gọi trực tiếp qua Netmind API (gồm GPT-OSS-120B và Qwen3.5-122B-A10B-FP8) thay vì triển khai cục bộ. Cấu hình mặc định để báo cáo kết quả là workflow ReAct kết hợp với GPT-OSS-120B; các workflow Plan & Execute và Reflexion được sử dụng làm đối chiếu để đánh giá ảnh hưởng của phương pháp tổ chức suy luận.

Thực nghiệm được tiến hành trên hai cấu hình: **Baseline** và **Evolved**. Trong đó, cấu hình **Evolved** tích hợp mô-đun **Procedural Memory** với tham số $top_k=5$, bộ nhớ giới hạn 1500 tokens, $skill_max_age=4$, $skill_pool=32$, bộ chọn LCB ($min_lcb=-0.05$), bộ điều khiển heuristic ($PP0_epsilon=0.05$), $nominee_k=3$ và số mẫu tiến hóa bằng 3. Mô-đun **Tool Refinement** được thiết lập với $num_docs=6$, $num_scoped_docs=4$, chiều dài tài liệu $doc_chars=500$, tần suất kiểm tra $planned_check=4$, $next_checks=2$ và ngưỡng hội tụ bằng 0.75 cho từng tiến trình chẩn đoán.

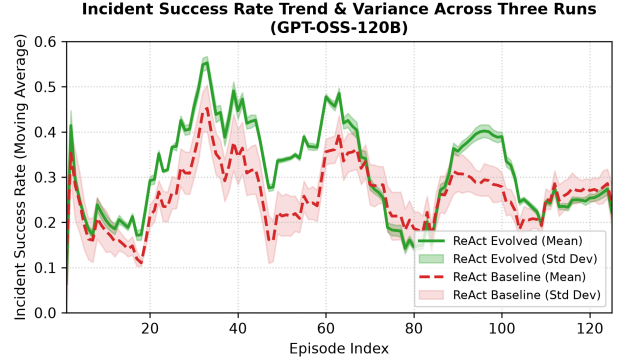
Bảng 2. Tóm tắt phạm vi benchmark trong thực nghiệm

Nhóm sự cố	Gốc	Tiến hóa	Tổng
Link Failures	7	5	12
End-Host Failures	9	7	16
Misconfigurations	14	13	27
Resource Contention	8	5	13
Network Node Errors	12	8	20
Network Under Attack	6	6	12
No-fault	0	0	25
Tổng	56	44	125

Tập benchmark con gồm 125 case được thiết kế theo dạng curriculum-evaluation: 100 case lỗi được sắp xếp theo trình tự tăng dần độ phức tạp để đánh giá sự tiến bộ của agent (gồm 44 biến thể tiến hóa chạy trước nhằm tích lũy kinh nghiệm, tiếp sau là 56 case gốc đại diện cho từng nguyên nhân sự cố); xen kẽ với đó là 25 case no-fault được phân bổ tất định để kiểm chứng khả năng nhận biết mạng bình thường.

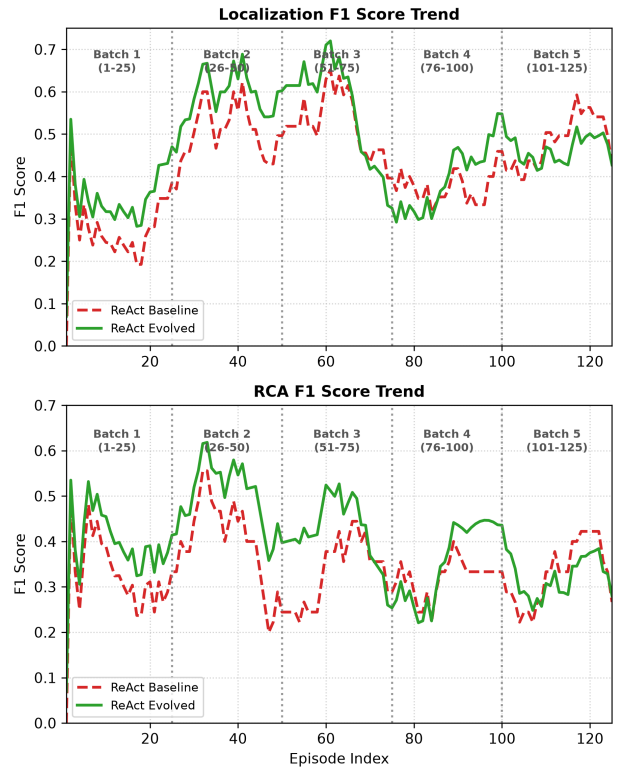
3.2. Kết quả thực nghiệm

Bảng 3 tổng hợp hiệu năng của workflow ReAct (chế độ Baseline và Evolved) trên hai backbone GPT-OSS-120B và Qwen3.5-122B-A10B-FP8. Hình 4 minh họa tiến trình học tập của backbone GPT-OSS-120B thông qua biến động chỉ số Localization F1 và RCA F1 theo từng batch 25 episode. Ngoài ra, Bảng 4 so sánh các



Hình 3. So sánh độ ổn định chỉ số Incident Success qua ba lần chạy của ReAct Baseline và ReAct Evolved trên GPT-OSS-120B.

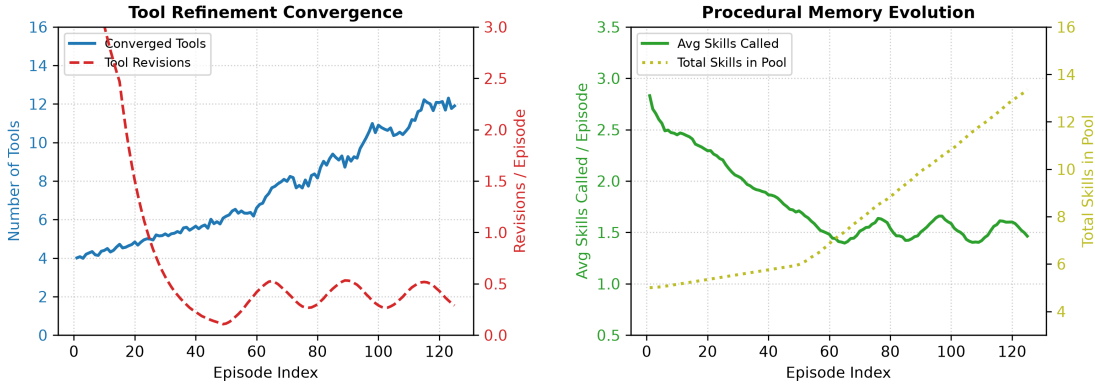
workflow ReAct, Plan&Execute và Reflexion trên cấu hình Evolved sử dụng backbone GPT-OSS-120B. Bên cạnh đó, Hình 5 theo dõi các chỉ số vận hành tiến hóa gồm mức độ hội tụ tài liệu, tần suất sửa đổi công cụ và trạng thái kích hoạt kỹ năng. Cuối cùng, Hình 3 thể hiện độ ổn định chỉ số Incident Success của cấu hình Evolved và Baseline qua ba lần chạy độc lập với backbone GPT-OSS-120B.



Hình 4. Diễn biến Localization F1 và RCA F1 theo từng batch của GPT-OSS-120B ở hai chế độ Baseline và Evolved.

Bảng 3. Kết quả chung của ReAct trên Baseline và Evolved với GPT-OSS-120B và Qwen3.5-122B-A10B-FP8

Metric	GPT-OSS Baseline	GPT-OSS Evolved	Qwen Baseline	Qwen Evolved
Detection	0.840	0.872 (+3.8%)	0.616	0.720 (+16.9%)
Localization Acc.	0.368	0.384 (+4.3%)	0.368	0.448 (+21.7%)
Localization F1	0.411	0.459 (+11.7%)	0.444	0.522 (+17.6%)
RCA Acc.	0.304	0.376 (+23.7%)	0.328	0.408 (+24.4%)
RCA F1	0.304	0.387 (+27.3%)	0.333	0.423 (+27.0%)
Incident Success	0.208	0.280 (+34.6%)	0.368	0.439 (+19.3%)
Avg. Steps	15.29	12.74 (-16.7%)	25.32	23.39 (-7.6%)
Avg. Tool Calls	13.60	11.09 (-18.5%)	41.98	37.55 (-10.6%)
Tool Error Rate	0.41%	0.00% (-100%)	0.32%	0.00% (-100%)
Input Tokens	15.30 M	17.27 M (+12.9%)	60.99 M	57.90 M (-5.1%)
Output Tokens	0.70 M	0.71 M (+1.8%)	0.69 M	0.67 M (-2.9%)
Total Tokens	16.00 M	17.98 M (+12.4%)	61.68 M	58.56 M (-5.1%)
Runtime	3.89 h	4.17 h (+7.2%)	5.71 h	6.29 h (+10.2%)



Hình 5. Xu hướng tiến hóa của hai mô-đun trong quá trình thực nghiệm: (a) Độ hội tụ tài liệu công cụ và tần suất sửa đổi mô tả của Tool Refinement; (b) Tần suất gọi kỹ năng và kích thước skill pool của Procedural Memory theo thời gian.

Bảng 4. So sánh hiệu năng và chi phí vận hành giữa các cấu trúc workflow trên cấu hình tiến hóa

Metric	ReAct	Plan&Execute	Reflexion
Detection	0.872	<u>0.890</u>	0.901
Localization	<u>0.459</u>	0.410	0.485
RCA	<u>0.387</u>	0.330	0.410
Incident Success	<u>0.280</u>	0.245	0.312
Input Tokens	<u>17.27 M</u>	13.82 M	23.83 M
Output Token	<u>0.71 M</u>	0.57 M	0.98 M
Runtime	<u>4.17 h</u>	3.34 h	5.75 h

4. ĐÁNH GIÁ HIỆU QUẢ

4.1. Tiêu chí đánh giá

Basic. Đo chất lượng chẩn đoán sự cố mạng của hệ thống thông qua các chỉ số về độ chính xác phát hiện thiết bị lỗi, định vị thực thể mạng bị ảnh hưởng và xác định nguyên nhân gốc rễ (RCA) trong các pha chẩn

đoán, đồng thời kết hợp đánh giá chi phí vận hành hệ thống như tài nguyên và thời gian chạy.

Evolve. Đánh giá năng lực tự thích ứng và cải tiến hành vi của tác nhân thông qua độ hội tụ của tài liệu hướng dẫn công cụ, tần suất kích hoạt và tái sử dụng các kỹ năng chuyên biệt trong bộ nhớ, cũng như mức độ giảm thiểu các thao tác chẩn đoán thừa.

4.2. Phân tích kết quả

Hiệu năng tổng thể. Kết quả thực nghiệm cho thấy việc tích hợp hai mô-đun tự tiến hóa giúp cải thiện rõ rệt các chỉ số chẩn đoán sự cố mạng, đặc biệt là độ chính xác phân tích nguyên nhân gốc rễ (RCA) trên cả hai mô hình nền. Cụ thể, điểm RCA Accuracy của GPT-OSS-120B tăng từ 0.304 lên 0.376 (cải thiện +23.7%), đồng thời chỉ số Incident Success tăng từ 0.208 lên 0.280 (tăng mạnh +34.6%). Đối với Qwen3.5-122B-A10B-FP8, điểm RCA Accuracy cải thiện từ 0.328 lên 0.408 (tăng +24.4%) và tỷ lệ xử lý sự cố thành công

Incident Success tăng từ 0.368 lên 0.439 (tương ứng cải thiện +19.3%). Bằng việc nạp thêm ngữ cảnh định hướng từ bộ nhớ quy trình và tài liệu công cụ đã tối ưu, hệ thống tác nhân đạt được chất lượng chẩn đoán vượt trội. Điều này khẳng định tri thức chẩn đoán được tích lũy liên tục qua các episode giúp nâng cao đáng kể năng lực suy luận chẩn đoán của hệ thống tác nhân mạng viễn thông so với cấu hình tính ban đầu.

Một số điểm đặc biệt. Thực nghiệm ghi nhận một số đặc điểm vận hành quan trọng: (i) việc áp dụng mô-đun tinh chỉnh tài liệu giúp đưa tỷ lệ gọi công cụ lỗi về mức tuyệt đối 0 phần trăm nhờ làm rõ các tham số và tiền điều kiện. Điều này giúp ngăn chặn LLM truyền sai đối số (arguments) hoặc gọi nhầm thiết bị không hỗ trợ công cụ chẩn đoán tương ứng; (ii) số lượng steps và tool calls được giảm bớt rõ rệt nhờ kế thừa trực tiếp các kỹ năng chẩn đoán tối ưu đã được đúc kết từ các lần chạy trước, cho phép agent khoanh vùng và loại trừ sớm các giả thuyết lỗi một cách có hệ thống thay vì dò quét tuần tự; (iii) lượng tokens tiêu thụ và runtime tổng thể có xu hướng gia tăng nhẹ ở một số cấu hình, đây là chi phí đánh đổi tất yếu do tác động của việc nạp thêm các kỹ năng chẩn đoán tương ứng từ bộ nhớ và tài liệu mô tả công cụ mở rộng vào ngữ cảnh prompt đầu vào của mô hình.

Mức độ hội tụ và sử dụng kỹ năng. Các chỉ số biểu diễn sự hội tụ tài liệu của **Tool Refinement** và quy mô skill pool của **Procedural Memory** (Hình 5) đạt kết quả tương đối tốt ở giai đoạn sau của thực nghiệm. Tuy nhiên, hiệu năng thực tế phụ thuộc nhiều vào việc thiết lập hyperparameter (như ngưỡng hội tụ tài liệu τ hay ngưỡng tương đồng bộ lọc kỹ năng), đòi hỏi phải thực hiện tinh chỉnh tham số nhiều lần để đảm bảo hệ thống tự thích ứng tốt hơn trên đa dạng các loại sự cố.

Thay đổi luồng hoạt động của tác nhân. Việc thử nghiệm hai mô-đun tự tiến hóa trên các workflow chẩn đoán khác nhau (Bảng 4) chỉ ra các cơ chế đánh đổi (trade-off) rõ rệt giữa chất lượng chẩn đoán và chi phí tài nguyên. Workflow Reflexion cho thấy độ chính xác RCA và tỷ lệ xử lý sự cố thành công (Incident Success) tốt nhất nhờ vòng lặp tự sửa sai động, tuy nhiên phải đánh đổi bằng chi phí tokens đầu vào cực kỳ lớn và thời gian chạy kéo dài. Ngược lại, Plan&Execute tối ưu hóa thời gian thực thi và lượng token tiêu thụ nhưng độ chính xác RCA và Incident Success suy giảm nghiêm trọng do thiếu tính phản ứng linh hoạt trước các phản hồi động của môi trường. Thực nghiệm chứng minh ReAct là workflow tối ưu và cân bằng nhất, đảm bảo tính động trong phản ứng mạng và giữ chi phí vận hành ở mức chấp nhận được. Kết quả này mở ra hướng đi đầy tiềm năng trong việc áp dụng các workflow lập luận hiện đại và linh hoạt hơn cho bài toán chẩn đoán tự động mạng viễn thông.

4.3. Hạn chế và rủi ro

Độ ổn định hệ thống (Hình 3). Do hạ tầng thực nghiệm kết hợp giữa thiết bị tính toán cá nhân và các API mô hình ngôn ngữ lớn từ bên thứ ba, tính bất định trong thời gian phản hồi và kết quả suy luận của tác nhân là khó tránh khỏi. Đồng thời, dưới các giới hạn nhất định về thời gian triển khai đề tài, các thí nghiệm đánh giá độ ổn định mới chỉ được thực hiện với 3 lần chạy lặp lại. Vì vậy, các chỉ số cải thiện thu được bước đầu nên được nhìn nhận như các tín hiệu tích cực mang tính định hướng, thay vì một kết luận thực nghiệm khẳng định hệ thống hoạt động tối ưu hoàn toàn. Việc kiểm chứng chặt chẽ hơn đòi hỏi thêm thời gian để thực hiện các đợt chạy lặp lại với số lượng mẫu lớn hơn trên mọi cấu hình, nhằm ước lượng chính xác khoảng dao động của các chỉ số hiệu năng trước khi đưa ra các khẳng định chính thức.

Quá tải cửa sổ ngữ cảnh. Việc tích hợp cả hai mô-đun tiến hóa tạo ra sự gia tăng lượng token trong prompt hệ thống. Ở mỗi bước chẩn đoán, tác nhân phải nạp thêm các kỹ năng quy trình tương ứng từ **Procedural Memory** (gồm điều kiện kích hoạt, quy trình thực thi và thiết bị mục tiêu), kết hợp với toàn bộ tài liệu mô tả công cụ đã được viết lại chi tiết của **Tool Refinement**. Đối với các incident phức tạp có chuỗi suy luận dài, việc nhồi nhét liên tục lượng tri thức công kênh này vào prompt qua nhiều lượt gọi dễ làm loãng sự chú ý của mô hình đối với các triệu chứng của thiết bị, hoặc vượt quá giới hạn cửa sổ ngữ cảnh cho phép.

Chi phí hội tụ và tối ưu hóa ngoại tuyến. Quy trình học của **Tool Refinement** yêu cầu chạy thử nghiệm mô phỏng trực tiếp trên các container Kathará, gây gánh nặng I/O và CPU lớn cho máy chủ. Thiết lập ngưỡng hội tụ τ không phù hợp dễ dẫn đến việc hội tụ chậm và lãng phí tài nguyên tính toán.

Khả năng tổng quát hóa và chuyển giao topology. Hình 4 cho thấy khả năng chuyển giao tri thức của evolved agent chưa thực sự ổn định. Ở Batch 1-2, **Skill Pool** và tài liệu công cụ được tinh chỉnh chính hiệu quả nhờ 44 case biến thể tiến hóa tương đồng cao về triệu chứng định tuyến và cấu hình thiết bị. Nhờ đó, agent nhanh chóng tích lũy các kỹ năng chẩn đoán phù hợp, thể hiện qua Gap RCA F1 đạt mức cao nhất +0.127 tại Batch 2. Tuy nhiên, khi chuyển sang các case gốc đa dạng hơn ở Batch 3-5, lợi thế này suy giảm rõ rệt và gần như biến mất ở Batch 5 (Gap RCA F1 chỉ còn -0.014). Kết quả này cho thấy các kỹ năng học được chủ yếu hiệu quả trong các cụm lỗi tương đồng, nhưng chưa chuyển giao tốt sang những tình huống tổng quát hơn hoặc topology khác biệt. Đây là hạn chế của **Procedural Memory**, đòi hỏi cơ chế tổng quát hóa kỹ năng tốt hơn để tránh quá khớp vào nhóm case ban đầu.

4.4. Hướng đi tương lai

Mở rộng và ứng dụng đa benchmark. Tiến hành đánh giá tác nhân trên các tập dữ liệu sự cố mạng thực tế từ doanh nghiệp hoặc các bộ benchmark chẩn đoán hệ thống phân tán khác ngoài NIKA để kiểm chứng toàn diện năng lực tổng quát hóa và độ tin cậy vận hành thực tế của tác nhân.

Huấn luyện trọng số mô hình nền. Cần kết hợp tối ưu hóa phi tham số với fine-tuning trực tiếp các trọng số của backbone (như LoRA/QLoRA) trên các trajectory thành công. Việc này giúp nội hóa tri thức mạng, tăng độ chính xác gốc và giảm tải ngữ cảnh.

Tiến hóa workflow của tác nhân. Phát triển các kiến trúc tự điều phối workflow động và cộng tác đa tác nhân (dynamic, multi-agent) thay vì sử dụng workflow đơn lẻ tĩnh. Cơ chế này cho phép phân rã sự cố phức tạp và ủy quyền chẩn đoán cho các tác nhân chuyên biệt (như chuyên gia phân tích cấu hình, chuyên gia giám sát lưu lượng) để xử lý hiệu quả các lỗi liên tầng.

5. KẾT LUẬN

Đề tài đã hoàn thành việc xây dựng và đánh giá thực nghiệm hệ thống tác nhân tự tiến hóa (Self-Evolving AI Agents) hỗ trợ chẩn đoán và xử lý sự cố mạng viễn thông trên nền tảng benchmark NIKA. Thông qua việc tích hợp hai mô-đun cốt lõi là **Procedural Memory** và **Tool Refinement**, hệ thống cho thấy năng lực tự cải tiến hành vi vượt trội qua các episode chẩn đoán thực tế. Kết quả thực nghiệm trên các mô hình ngôn ngữ lớn làm backbone gồm GPT-OSS-120B và Qwen3.5-122B-A10B-FP8 chứng minh hướng đi này giúp nâng cao đáng kể độ chính xác xác định nguyên nhân gốc rễ (RCA), đưa tỷ lệ lỗi gọi công cụ về mức tuyệt đối và tối ưu hóa đáng kể số bước suy luận mà không cần tái huấn luyện mô hình. Tính mới của đề tài nằm ở việc định hình kiến trúc tác nhân như một thực thể động có khả năng tự sửa lỗi và tích lũy tri thức có cấu trúc từ các vết lỗi trong quá trình tương tác thực tế với môi trường mạng viễn thông.

Về mặt đóng góp cá nhân, sinh viên đã trực tiếp thực hiện toàn bộ quy trình từ nghiên cứu lý thuyết về các phương pháp tự tiến hóa, thiết kế kiến trúc tích hợp với nền tảng NIKA, hiện thực mã nguồn cho hai mô-đun học kinh nghiệm, thiết lập pipeline thực nghiệm tự động hóa và phân tích số liệu kết quả. Trong các bước phát triển tiếp theo, hệ thống hướng tới việc tối ưu hóa khả năng chọn lọc và pruning bộ nhớ để tránh lãng phí chi phí token, đồng thời nghiên cứu khả năng thích ứng linh hoạt trên nhiều topology mạng thực tế quy mô lớn hơn. Khi được hoàn thiện, giải pháp này hứa hẹn sẽ trở thành một công cụ trợ lý thông minh đặc lực cho các kỹ sư vận hành hệ thống mạng, giúp tự động hóa

quy trình thu thập bằng chứng, khoanh vùng sự cố và chuẩn hóa các báo cáo kỹ thuật một cách nhanh chóng và chính xác.

TÀI LIỆU THAM KHẢO

- [1] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker. P4: Programming protocol-independent packet processors. In *ACM SIGCOMM Computer Communication Review*, volume 44, pages 87–95, 2014. URL <https://arxiv.org/abs/1312.1719>. 2
- [2] H. Chase et al. LangGraph: Building stateful, multi-actor applications with LLMs. <https://github.com/langchain-ai/langgraph>, 2024. 2
- [3] FRRouting Project. FRRouting: A free range routing protocol suite. <https://github.com/FRRouting/frr>, 2021. 2
- [4] H.-a. Gao, J. Geng, W. Hua, et al. A survey of self-evolving agents. *Transactions on Machine Learning Research*, 2026. URL <https://arxiv.org/abs/2507.21046>. 1
- [5] S. Marrone, L. Nicoletti, and C. Pizzuti. Kathará: a container-based framework for network emulation. In *IEEE International Conference on Computer Communication and Networks*, pages 1–9, 2019. URL <https://github.com/KatharaFramework/Kathara>. 2
- [6] Q. Mi, Z. Ma, M. Yang, H. Li, Y. Wang, H. Zhang, and J. Wang. Skill-pro: Learning reusable skills via non-parametric PPO for LLM agents. In *International Conference on Machine Learning*, 2026. URL <https://arxiv.org/abs/2602.01869>. 1, 2
- [7] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen. DRAFT: Tool mastery via self-driven interactions. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.08197>. 1, 3
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. URL <https://arxiv.org/abs/1707.06347>. 3
- [9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>. 2
- [10] Z. Wang, A. Cornacchia, A. Sacco, F. Galante, M. Canini, and D. Jiang. A network arena for benchmarking AI agents on network troubleshooting. *arXiv preprint arXiv:2512.16381*, 2025. URL <https://arxiv.org/abs/2512.16381>. 1, 2
- [11] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>. 2